

AI Travel Planner

Pietro De Angeli

I. INTRODUCTION

A. Dialogue System Description

The AI Travel Planner is a task-oriented dialogue system designed to assist users in planning travel through natural conversation, similar to interacting with a travel agency. Existing systems are typically based on form-filling or rigid slot-based interactions, which can be cumbersome for users with varying levels of informativeness and experience.

Our system leverages the flexibility of large language models (LLMs) while maintaining predictable control through a modular architecture and constrained action schemas. Responses are designed to be engaging and natural, incorporating grounding signals and proactive suggestions.

Users can plan trips through three main booking intents: flight reservations, accommodation bookings, and activity planning, as well as informational queries such as city comparison. Out-of-domain requests are handled through clarification prompts, while slot carryover is supported across related bookings to reduce repetition. However, the user can reject the carryover keeping independent information among different bookings.

The system accommodates both under-informative users, who require progressive slot elicitation, and over-informative users, who may provide complete information or multiple requests within a single utterance. The exchanges are mixed-initiative, allowing the system to proactively suggest slot values or follow-up intents while still respecting user control.

Responses include short grounding acknowledgments at each turn and a final summary with confirmation once all required slots are filled. Until completion, the system focuses on requesting missing information and validating newly provided values.

B. Target Users

The primary users are travelers planning trips who prefer conversational interaction over form-filling interfaces. The system accommodates both novice users, who need guidance throughout the booking process, and experienced travelers, who tend to provide comprehensive information upfront.

II. CONVERSATION DESIGN

A. Dialogue Types

The system supports **task-oriented dialogues** aimed at completing specific goals such as booking or planning activities. These dialogues collect, validate, and confirm required information before task completion.

The system implements a **mixed-initiative dialogue** model. Both the user and the system can take control of the conversation flow:

- Users may freely choose which intent to pursue; the segmenter tracks the active intent and passes it to the dialogue manager.
- The system may request missing information, guide the interaction, or suggest follow-up intents.

The system supports **underinformative** users, who provide incomplete information and require progressive slot elicitation, and **overinformative** users, who may provide multiple slot values or multiple intents within the same utterance.

The system supports **multiple intents** within the same utterances segmented by the Intent Splitter and processed sequentially.

The system handles **meta-dialogue** actions such as confirmation sub-dialogues, clarification requests, and dialogue termination.

B. Interaction Properties

Initiative and Engagement: The system follows a mixed-initiative strategy. Users may provide multiple slots or switch intents at any time, while the system proactively requests missing information or suggests follow-up intents.

Confirmation Strategy: Explicit confirmation is required before booking completion. The system, after collecting all the necessary slots, summarizes them and awaits user approval. Negative confirmation triggers controlled slot revision.

Grounding and Acknowledgment: Each user contribution is acknowledged through concise grounding signals.

Fallback and Error Recovery: Out-of-domain inputs trigger clarification prompts. Unrecognized intents default to ASK_CLARIFICATION, preserving dialogue continuity.

Slot Carryover: For related bookings, previously collected slots (e.g., destination, dates) can be reused upon explicit user acceptance, reducing redundancy while preserving user control.

III. CONVERSATION MODEL

A. Pipeline Architecture

The conversational system is built upon **Llama 3.1 8B Instruct**, PyTorch and Python 3.10.0. The same backbone model is reused across components, while architectural modularity is enforced through structured prompting and explicit dialogue state passing.

The pipeline consists of four modules: Intent Splitter, Natural Language Understanding (NLU), Dialogue Manager (DM), and Natural Language Generation (NLG).

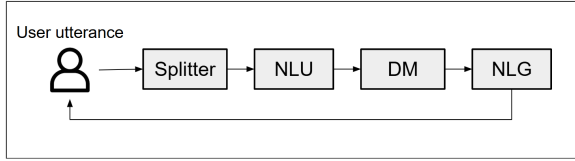


Fig. 1. Modular architecture of the AI Travel Planner pipeline.

Each module is steered through component-specific prompts while sharing a common dialogue state representation.

The NLU prompts are conditioned by the dialogue state to reduce ambiguity and increase robustness, especially in control turns such as confirmation, denial, and slot revision.

Two versions of the pipeline were implemented: a deterministic rule-based DM and a LLM-based DM. The rule-based version provides more stable transitions, while the LLM-based DM is more flexible and scalable to more complex dialogue flows. The Evaluation section compares both approaches.

B. Component Details

The components are designed to work together while maintaining clear functional boundaries:

- **Intent Splitter:** Detects multi-intent inputs, separates them, and queues remaining intents for subsequent turns.
- **NLU:** Produces structured JSON containing intent and slot values. A prompt helper (`state_context`) injects the latest dialogue state to constrain interpretation and reduce ambiguity. **Grounding** has been introduced before creating the JSON to improve stability of the module's result
- **DM:** Selects the next system action from a predefined action schema. Outputs are validated against schema constraints and are parameterized (e.g., `REQUEST_MISSING_SLOT(slot)`).
- **API tool:** Executes external API calls based on DM actions and updates the dialogue state with retrieved information. It uses Amadeus APIs for accommodation, flights and activity retrieval, and OpenStreetMap Nominatim for geocoding city names.
- **NLG:** Generates concise text-conditioned responses based on the selected action and the current dialogue state snapshot. When needed, it also summarizes the dialogue state before confirmation to reduce misunderstandings.

C. Intents and Slots

The full list of intents and required slots is reported in Appendix A. Although some slots share names across intents (e.g., `destination`, `budget_level`), values are stored independently per booking and reused only through explicit carryover.

Intent Schema Design: A single generic intent (“travel planning”) introduced ambiguity in intent classification and slot extraction. The system therefore adopts separated intents (`BOOK_FLIGHT`, `BOOK_ACCOMMODATION`, `BOOK_ACTIVITY`, `COMPARE_CITIES`) to reduce semantic overlap and improve extraction reliability.

D. Prompt Strategy

Each module is controlled through a dedicated prompt designed to constrain the model output while preserving dialogue context.

Structured outputs. The NLU and LLM-based DM are constrained to return fixed JSON schemas (e.g., intents, slots, actions). This avoids fragile free-text parsing and simplifies dialogue-state updates.

State-conditioned NLU. The NLU prompt is dynamically adapted according to the last DM action. Specialised prompt variants are used for confirmation handling and slot revision, while the base prompt injects the supported intents, slot schemas, and domain constraints. Dialogue-state grounding reduces ambiguity in short or context-dependent user utterances.

One-shot prompting. An example is included for confirmation, slot-change handling, and multi-intent splitting in order to stabilise the expected JSON format and improve robustness across different surface forms.

Rule-constrained DM. The LLM-based DM prompt contains explicit priority rules and action constraints. The model receives the serialized dialogue state, missing slots, and carryover metadata, grounding action selection in the current dialogue context.

Action-conditioned NLG. The NLG module uses different prompt templates depending on the selected DM action (e.g., slot request, confirmation summary, clarification, goodbye). Only the relevant subset of the dialogue state is injected to keep prompts concise.

Dialogue history and temporal grounding. The last dialogue turns and the current date are injected into the prompts, enabling context-aware responses and interpretation of relative temporal expressions such as “next Monday” or “in two weeks”.

IV. EVALUATION

A. Data Description and Analysis

1) *Data Collection Method:* The evaluation datasets were generated with GPT-5.2, prompted to create diverse structured utterances covering all supported intents and slot combinations. The test set includes under-informative, over-informative, and multi-intent cases to evaluate robustness across different user behaviors.

2) Dataset Characteristics:

- **NLU Test Set:** 90 structured utterances covering all supported intents, including 75 **single-intent** and 15 **multi-intent** utterances. Each test case includes annotated ground-truth intents and slot values.
- **DM Test Set:** 40 dialogue-state snapshots with corresponding expected next actions across different dialogue contexts and complexity levels. The *ground-truth* action has been obtained using the rule-based DM.
- **Pipeline Evaluation:** 16 scripted dialogues simulating realistic interactions.
- **NLG:** evaluated through the pipeline and human evaluation.

3) NLU Dataset Analysis:

- **Intent Distribution:** The 75 **single-intent** utterances are balanced across the four booking intents (≈ 18 cases each), while OOD accounts for 15 cases and END_DIALOGUE for 5.
- **Complexity Levels:** 3 complexity levels are defined by the `information_type` field: *under_informative* (one or a few slots provided), *over_informative* (many or all information provided), and *noisy_paraphrase* (informal or highly variable wording).
- **Multi-Intent Cases:** 15 utterances contain multiple concurrent intents. Of these, 11 combine two intents, while 4 chain three intents.

B. Evaluation Metrics

1) *Intrinsic Evaluation:* The intrinsic evaluation focuses on the performance of individual components (NLU, DM, Intent Splitter) using the following metrics:

- **Intent Classification Accuracy:** Percentage of correctly classified intents.
- **Slot Extraction F1:** Precision, recall, and F1 for slot-value extraction.
- **DM Action Accuracy:** Percentage of correct action selections.
- **Splitter Split Accuracy:** Percentage of correctly segmented multi-intent inputs.

2) *Extrinsic Evaluation:* The extrinsic evaluation assesses overall system performance in realistic dialogue scenarios with a **pipeline evaluation** (a dialogue is considered successful if it reaches the expected completion) and **human evaluation** with a set of evaluation questions provided to a small group of users.

C. Results

1) *NLU Evaluation:* Table I compares the NLU module with and without dialogue-state grounding across 95 evaluated items.

TABLE I
NLU OVERALL PERFORMANCE

Config	Intent Acc.	Exact Match	Slot F1
w/ grounding	91.7%	85.6%	93.1%
w/o grounding	90.0%	54.6%	74.1%

Grounding provides a small gain in intent accuracy, increasing from 90.0% to 91.7%, while substantially improving slot extraction. Exact match rises from 54.6% to 85.6%, and Slot F1 increases from 74.1% to 93.1%. This shows that dialogue-state grounding improves slot consistency, especially when user utterances contain ambiguous or multiple slot values.

Table II reports intent accuracy and exact match broken down by complexity tier for single-intent utterances.

Grounding improves performance across all complexity tiers, particularly for slot extraction.

For *under_informative* utterances, intent accuracy remains perfect in both settings, while exact match improves from 81.5% to 92.6%.

TABLE II
NLU PERFORMANCE BY COMPLEXITY TIER (SINGLE-INTENT UTTERANCES)

Type	Intent (G)	Exact (G)	Intent (NG)	Exact (NG)
under_informative	100.0%	92.6%	100.0%	81.5%
over_informative	95.0%	95.0%	90.0%	40.0%
noisy_paraphrase	80.0%	69.2%	80.0%	42.3%

The largest gain is observed for *over_informative* utterances, where exact match increases from 40.0% to 95.0%, showing that dialogue-state grounding helps resolve ambiguity when many slot values are provided in a single utterance.

Noisy paraphrase utterances remain the most challenging category, with intent accuracy limited to 80.0% in both settings. However, grounding still improves exact match from 42.3% to 69.2%.

2) *Intent Splitter Evaluation:* The Intent Splitter was evaluated by comparing the predicted number of queued intent segments with the expected decomposition of each utterance.

The splitter achieved 92.3% exact queue-match accuracy (12/13 multi-intent cases), with a Mean Absolute Queue Error (MAQE) of 0.15. The only failure occurred on a dense three-intent request, where all intents were merged into a single segment.

3) *Dialogue Manager Evaluation:* The LLM-based DM was evaluated on 40 dialogue-state snapshots. Table III reports the per-action F1 scores.

TABLE III
LLM DM ACTION CLASSIFICATION (40 TEST CASES)

Action	F1	n
REQUEST_MISSING_SLOT	0.81	19
ASK_CONFIRMATION	0.77	6
REQUEST_SLOT_CHANGE	0.60	4
ASK_CLARIFICATION	1.00	4
COMPARE_CITIES_RESULT	0.67	2
GOODBYE	1.00	2
COMPLETE_FLIGHT_BOOKING	1.00	1
COMPLETE_ACCOMMODATION_BOOKING	1.00	1
COMPLETE_ACTIVITY_BOOKING	0.00	1
OFFER_SLOT_CARRYOVER	0.50	1
Overall accuracy	75.6%	
Macro F1	73.5%	

Terminal actions achieve perfect scores. The main failures involve premature COMPARE_CITIES_RESULT actions, incorrect carryover transitions, and post-confirmation intent switches.

4) *Pipeline Evaluation:* The end-to-end pipeline was evaluated using the LLM-based DM with Intent Splitter enabled on 16 scripted dialogues.

The pipeline achieves 75% task success (12/16 dialogues). BOOK_ACCOMMODATION and END_DIALOGUE achieve 100% success, while failures mainly occur in under-informative interactions and highly detailed narrative requests.

D. Human Evaluation

Following the human-evaluation guidelines, I ran a small pilot with ten volunteers (informal crowdsourcing). Each participant interacted with the system twice. In condition A

(*cold start*), they received no system description. In condition B (*briefed*), they were given little instructions describing how to use correctly the system (book a trip using flights/hotels/activities), the expected confirmation steps, and the supported intents.

After each interaction, participants answered four 1–5 questions (1=Completely disagree, 5=Completely agree):

- 1) Did the system complete the task?
- 2) Did it ask for missing information/clarifications when needed?
- 3) Were responses natural and coherent and engaging?
- 4) Was the system polite?

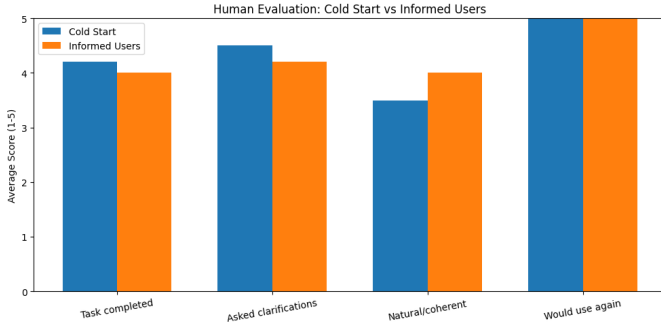


Fig. 2. Comparison of human-evaluation scores between cold-start and briefed users.

The results show that the system performed well in both settings. Task completion and clarification handling achieved high scores in both conditions, suggesting that the dialogue manager was generally able to recover missing information correctly.

Interestingly, the *cold start* setting obtained slightly higher scores for task completion and clarification handling. A possible explanation is that uninformed users interacted more naturally and incrementally with the system, allowing the dialogue manager to guide the conversation more effectively.

The *briefed* users instead reported higher scores for response naturalness and coherence, likely because they better understood the system capabilities and interaction flow.

Finally, politeness achieved the maximum score in both conditions, indicating that the system maintained a consistent and user-friendly conversational style.

V. DISCUSSION

The evaluation highlights the importance of grounding the NLU with the dialogue state. Providing contextual information significantly improved slot extraction accuracy since the model could better infer the expected semantic meaning of the user input. In particular, grounding mainly improved slot consistency and exact-match accuracy, while intent recognition was already relatively stable even without contextual information.

Finally, the rule-based dialogue manager showed more stable behavior than the LLM-based version because the transitions were explicitly defined through handcrafted rules and slot-completion checks. However, the LLM-based dialogue manager is potentially more scalable and adaptable to unseen dialogue states and future extensions of the system.

The error analysis also highlighted some recurrent failure modes. The most common issues emerged in complex multi-intent requests, where the intent splitter occasionally failed to correctly separate chained tasks. Additional errors were caused by semantic overlap between intents, such as confusion between city-comparison requests and activity-booking queries. These results suggest that handling long and highly compositional user requests remains one of the main challenges for modular conversational systems.

VI. LIMITATIONS

- **Limited intent and slot coverage.** Only four booking intents are supported; real-world needs such as cancellations, multi-leg trips, or car rental are not handled.
- **LLM inference cost.** Each turn requires 4 LLM forward passes, resulting in high latency without GPU acceleration.
- **Small-scale evaluation.** The pipeline was tested on 16 static dialogues and the human pilot involved only 10 volunteers, limiting statistical significance.
- **No long-conversation stress test.** All evaluated dialogues are short; behaviour on extended sessions (20+ turns) with topic switches and corrections remains untested.

VII. CONCLUSION

The AI Travel Planner is a modular task-oriented dialogue system powered by Llama 3.1 8B Instruct, supporting flight, accommodation, and activity booking through a four-stage pipeline (Intent Splitter, NLU, DM, NLG). Dialogue-state grounding proved highly beneficial, raising Slot F1 from 74.1% to 93.1%. The end-to-end pipeline achieves good results in the human evaluation. Future work includes broader intent coverage, lower inference latency, and larger-scale user evaluation. Overall, the project demonstrates how modular LLM-based architectures can combine conversational flexibility with controlled dialogue management in task-oriented settings.

REFERENCES

- [1] Meta AI, “LLaMA 3.1 8B Instruct, . [Online]. Available: <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>
- [2] Amadeus IT Group, Amadeus for Developers, [Online]. Available: <https://developers.amadeus.com/>
- [3] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems* 32. Curran Associates, Inc., 2019, pp. 8024–8035. <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>

APPENDIX A INTENT AND SLOT DEFINITIONS

A. Intents

The system supports the following intents:

- **BOOK_FLIGHT**: User wants to book or search for flights. This includes specifying origin, destination, dates, number of passengers, and budget preferences.
- **BOOK_ACCOMMODATION**: User wants to book hotels, hostels, or other lodging. Requires destination, check-in/check-out dates, number of guests, and budget level.
- **BOOK_ACTIVITY**: User wants to book tours, experiences, or activities at a destination. Requires destination, activity category (e.g., cultural, adventure, food), preferred time and budget level.
- **COMPARE_CITIES**: User wants to compare two cities for specific activities. An informational intent that does not result in a booking.
- **END_DIALOGUE**: User wants to end the conversation (e.g., "goodbye", "thanks, bye").
- **OOD** (Out of Domain): Requests not related to travel booking or that cannot be understood by the system.

B. Slots

Table IV lists all slots used across intents, their descriptions, and applicable intents.

TABLE IV
SLOT DEFINITIONS

Slot Name	Description	Intents
origin	Departure city/airport	BOOK_FLIGHT
destination	Target city or place	BOOK_FLIGHT, BOOK_ACCOMMODATION, BOOK_ACTIVITY
departure_date	Flight departure date (YYYY-MM-DD)	BOOK_FLIGHT
return_date	Flight return date (optional)	BOOK_FLIGHT
num_passengers	Number of flight travelers	BOOK_FLIGHT
check_in_date	Hotel check-in date (YYYY-MM-DD)	BOOK_ACCOMMODATION
check_out_date	Hotel check-out date (YYYY-MM-DD)	BOOK_ACCOMMODATION
num_guests	Number of hotel guests	BOOK_ACCOMMODATION
activity_category	Activity type: adventure, cultural, food, sport, relax, nature, nightlife, family	BOOK_ACTIVITY, COMPARE_CITIES
preferred_time	Preferred time of day: morning, afternoon, evening	BOOK_ACTIVITY
budget_level	Budget preference: low, medium, high	BOOK_FLIGHT, BOOK_ACCOMMODATION, BOOK_ACTIVITY
city1	First city for comparison	COMPARE_CITIES
city2	Second city for comparison	COMPARE_CITIES

C. Activity Categories

For **BOOK_ACTIVITY** and **COMPARE_CITIES** intents, the `activity_category` slot accepts the following values:

- **adventure**: hiking, trekking, climbing, kayaking, biking, outdoor activities
- **cultural**: museums, galleries, cathedrals, monuments, heritage sites
- **food**: culinary tours, wine tastings, cooking classes, food markets
- **sport**: stadiums, football matches, sporting events
- **relax**: spa, wellness, cruises, panoramic tours
- **nature**: parks, gardens, scenic landscapes, nature walks
- **nightlife**: bars, clubs, concerts, live music
- **family**: zoos, aquariums, theme parks, family-friendly activities
- **general**: unspecified or mixed activities

APPENDIX B EXAMPLE DIALOGUES FROM PIPELINE EVALUATION

The following dialogues show representative examples collected during the pipeline evaluation. The examples illustrate both over-informative and under-informative interaction styles, together with the corresponding internal module outputs.

A. Flight Booking — Over-Informative (2 turns)

Corresponds to `flight_full_static`. The user supplies all required slots in one utterance.

USR Book a flight from Rome to Paris on 2026-08-14 for 2 passengers, medium budget.

NLU `intent: BOOK_FLIGHT, slots: {origin: Rome, destination: Paris, departure_date: 2026-08-14, num_passengers: 2, budget_level: medium}`

DM `ASK_CONFIRMATION`

SYS Perfect! Here is a summary of your flight: Rome → Paris on August 14th, 2 passengers, medium budget. Shall I proceed with this booking?

USR Yes, confirm.

NLU `slots: {confirmation: yes}`

DM `COMPLETE_FLIGHT_BOOKING`

SYS Excellent! Your flight from Rome to Paris has been booked. Is there anything else I can help you with?

B. Flight Booking — Incremental (6 turns)

Corresponds to `flight_incremental_static`. The user provides one slot per turn (under-informative).

USR I want to fly to Barcelona.
 NLU intent: BOOK_FLIGHT, slots: {destination: Barcelona}
 DM REQUEST_MISSING_SLOT(origin)
SYS Great, Barcelona is a wonderful choice! Where will you be departing from? Popular options are Rome, Paris, and London.
USR From Milan.
 NLU slots: {origin: Milan}
 DM REQUEST_MISSING_SLOT(departure_date)
SYS Got it, departing from Milan. When would you like to fly?
USR 2026-09-01.
 NLU slots: {departure_date: 2026-09-01}
 DM REQUEST_MISSING_SLOT(num_passengers)
SYS September 1st, noted! How many passengers?
USR 3 people.
 NLU slots: {num_passengers: 3}
 DM REQUEST_MISSING_SLOT(budget_level)
SYS Almost there! What's your budget preference — low, medium, or high?
USR High.
 NLU slots: {budget_level: high}
 DM ASK_CONFIRMATION
SYS Here's the summary: Milan → Barcelona on 2026-09-01, 3 passengers, high budget. Should I confirm?
USR That's correct.
 NLU slots: {confirmation: yes}
 DM COMPLETE_FLIGHT_BOOKING
SYS All set! Your flight has been booked. Need anything else?

C. Accommodation Booking — Over-Informative (2 turns)

Corresponds to hotel_full_static.

USR Book a hotel in Amsterdam from 2026-07-20 to 2026-07-25 for 2 guests, low budget.
 NLU intent: BOOK_ACCOMMODATION, slots: {destination: Amsterdam, check_in_date: 2026-07-20, check_out_date: 2026-07-25, num_guests: 2, budget_level: low}
 DM ASK_CONFIRMATION
SYS Perfect! Accommodation in Amsterdam, check-in July 20th, check-out July 25th, 2 guests, low budget. Shall I book it?
USR Looks good!
 NLU slots: {confirmation: yes}
 DM COMPLETE_ACCOMMODATION_BOOKING
SYS Done! Your hotel in Amsterdam has been reserved. Can I help with anything else?

D. City Comparison (1 turn)

Corresponds to compare_cities.

USR Compare Vienna and Prague for cultural activities.
 NLU intent: COMPARE_CITIES, slots: {city1: Vienna, city2: Prague, activity_category: cultural}
 DM COMPARE_CITIES_RESULT
SYS Both cities are excellent for culture! Vienna stands out for its imperial palaces and classical music heritage, while Prague offers stunning medieval architecture and vibrant gallery scene. Would you like to book activities in either city?

E. End Dialogue (1 turn)

Corresponds to end_dialogue_static.

USR Thanks, bye!
 NLU intent: END_DIALOGUE
 DM GOODBYE
SYS Thank you for using the travel planner — have a wonderful trip! Goodbye!

APPENDIX C

IMPROVEMENTS FROM PREVIOUS SUBMISSION

- 1) **Bug fixes and codebase cleanup.** Several bugs in the dialogue flow were resolved and the entire codebase was refactored for readability and maintainability.
- 2) **NLU decoupled from DST.** The NLU module no longer accesses the dialogue state directly. Instead, a lightweight context prompt is assembled externally (in the main loop) and injected into the NLU call, making the module fully self-contained and easier to test in isolation.
- 3) **Introduced new slots.** To make the system look more realistic new slots have been introduced (origin for flight booking and preferred_time for the activities).
- 4) **Mixed-initiative enhanced.** The NLG now proactively suggests slot values and possible options (e.g., destinations, budget levels, activity categories) when requesting missing information, resulting in a more engaging and guidance-oriented interaction.
- 5) **Dialogue history passed to NLG.** The last turns of the conversation history are now forwarded to the NLG module, enabling more context-aware responses such as acknowledging corrections or referencing earlier user statements.
- 6) **NLU Grounding:** Introduced grounding after NLU's JSON generation to improve the stability of its outputs.